

Retrofitting Pretrained Transformers with Progressive Retrieval Attention: Thin Hugging Face Adapters and a Qwen Correctness Study

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 11, 2026

Abstract

Progressive Retrieval Attention (PRA) selects sparse, layer-native key/value (K/V) memory from references or displaced prompt history. Controlled models establish the mechanism but do not show that it can preserve a real pretrained transformer’s attention semantics. We begin a pretrained integration study with a shared PRA execution core and thin Hugging Face attention adapters. The first adapter wraps one upper layer of the frozen 596M-parameter **Qwen/Qwen3-0.6B** checkpoint. When memory is disabled, the wrapper delegates to the original module and obtains bit-exact logits, hidden states, greedy generation, and all 28 generation-cache shapes. When memory is enabled, it reuses Qwen’s pretrained projections, Q/K normalization, rotary embedding, grouped-query layout, causal mask, cache update, eager attention function, and output projection. Stored memory retains eight native K/V heads rather than expanding to 16 query heads. A CUDA smoke run materializes reference K/V from CPU and executes a 408-token logical prompt under a 256-token native-operation bound. A subsequent evidence-ranking study separates routing from materialization: post-RoPE key means, pre-RoPE key means, and attention-input hidden-state means all retain the same post-RoPE detail K/V. On 16 matched HotpotQA/QASPER examples with 32-token chunks, post-RoPE routing has a strong late- position correlation ($r = 0.652$) and recall@3 of 0.125. Pre-RoPE routing removes that correlation ($r = 0.009$), while hidden-state routing gives the best sparse recall@3, 0.438. Two confirmation subsets totaling 64 evaluations give hidden-state recall@3/8/16 of 0.391/0.578/0.797, but QASPER recall@3 is only 0.156 and recall@16 selects 23.8% of chunks. Thus semantic selection and positional K/V transport should be separated, but zero-parameter Qwen routing does not pass the predeclared gate for Llama integration.

1 Introduction

Pretrained decoder families differ in projection layout, rotary implementation, grouped-query attention (GQA), masks, sliding windows, and generation-cache conventions. Replacing a complete model class to add memory duplicates rapidly changing library code and makes parity difficult to audit. The alternative studied here is narrower:

normal pretrained model $\rightarrow$ wrap selected attention modules $\rightarrow$ shared PRA core.

The family adapter owns only native attention semantics. The shared core owns references, chunk gists, exact routing, the materialization budget, selected K/V transfer, and metrics. This boundary follows the broader transformer interface of queries attending to keys and values [9],

while preserving Qwen’s pretrained RoPE and GQA choices rather than approximating them in a parallel implementation [8, 1, 7].

This paper begins with Qwen because a sub-billion-parameter checkpoint makes exact tests practical on constrained hardware while still exercising modern RoPE and GQA. It does not yet claim cross-family portability or useful long-context QA. The contribution is a tested first adapter and an evidence hierarchy that prevents working transport from being mistaken for working retrieval.

2 PRA Recap and Prior Contracts

For layer ℓ , PRA stores reference chunks as native $K_c^{(\ell)}, V_c^{(\ell)} \in \mathbb{R}^{H_{kv} \times T_c \times d_h}$ and one or more compact gists $g_c^{(\ell)}$. A query representation $r^{(\ell)}$ scores the gists, and a global budgeter selects whole chunks. Materialized memory precedes the direct K/V sequence under one softmax:

$$\text{Attn}(Q, [K_M; K_D], [V_M; V_D]) = \text{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M\right) [V_M; V_D]. \quad (1)$$

This is not a separately normalized cross-attention residual. The native output projection is applied once to the combined result.

Paper 1 established controlled native-K/V routing and materialization. Paper 1.5 separated logical source position from contextual fragmentation. Continuous sources use $p_i = p_{\text{logical start}} + i$, and ordinary post-RoPE keys remain the canonical cache state. The present adapter preserves that contract by capturing each selected HF layer’s actual post-RoPE keys and by assigning source-relative positions across bounded encoding blocks.

3 Adapter Architecture

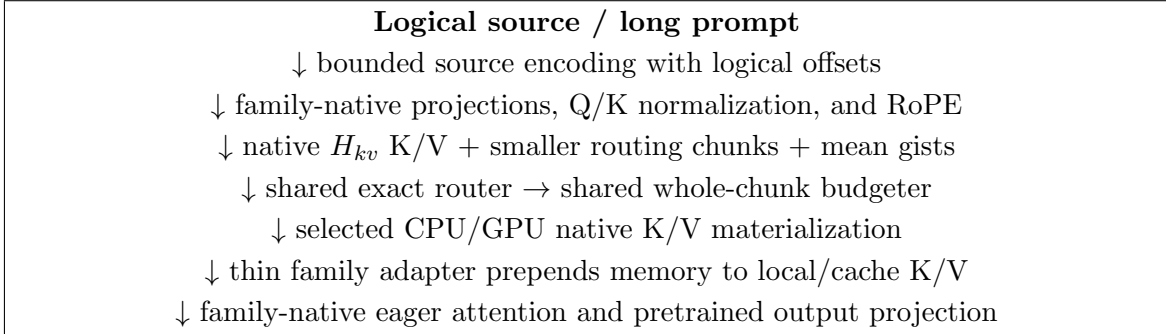


Figure 1: Paper 2 boundary. Model-family semantics stay in a thin adapter; references, routing, budgeting, materialization, residency, and metrics stay in one shared PRA core.

3.1 Shared execution core

The controlled model’s former monolithic attention forward pass was divided into reusable operations. `prepare_pra_query` selects the newest valid query and maps GQA query groups into the native K/V routing width. `route_memory` calls the exact packed-index router. `budget_selected_memory` enforces

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{native, max}}. \quad (2)$$

`materialize_selected_kv` removes duplicate overlap, preserves row isolation, and moves only retained detail K/V. `combine_local_and_memory_kv` pads variable rows and prepends an additive all-visible memory mask to the family’s existing local causal mask. The controlled model and HF adapters now call this same core.

3.2 Thin HF contract

The abstract adapter exposes six family operations: project Q/K/V, apply native position encoding, normalize tensor layout, combine the native mask, invoke PRA attention, and project the output. Disabled memory takes an even narrower path: call the original attention object unchanged. This design preserves pretrained parameters and avoids checkpoint conversion.

The Qwen adapter has 272 physical lines (248 nonblank, non-comment lines). It introduces no learned parameter. Its family-specific branches distinguish Qwen2-style projections from Qwen3’s Q/K normalization; routing, budgeting, materialization, residency, and metrics remain outside the file. This is a portability measurement, not a code-minimization target.

4 Qwen Native Semantics

4.1 Projection and RoPE

The adapter calls the wrapped module’s `q_proj`, `k_proj`, and `v_proj`. Qwen3’s existing `q_norm` and `k_norm` are applied before the installed Transformers `apply_rotary_pos_emb` function. Reference capture saves post-RoPE K and position-independent V . No independent RoPE implementation is introduced.

4.2 Grouped-query attention

The tested checkpoint has $H_q = 16$, $H_{kv} = 8$, and $d_h = 128$. Cache objects retain $[B, 8, T, 128]$ K/V. For routing only, each pair of query heads sharing one K/V head is averaged, producing a vector in the same 8×128 space as a flattened key gist. K/V expansion occurs inside Qwen’s eager attention function, never in persistent PRA storage. A synthetic replay test compares this path with explicit K/V head repetition and matches within 10^{-6} .

4.3 HF cache and masks

Prefill and single-token decode both use the wrapped module’s HF `Cache.update` call. PRA memory is prepended after that update, so direct generation history appears once. A decode test with four prompt tokens and two generated tokens observes five direct cache positions at the last invocation, plus four separate memory positions. The original causal mask remains on local/cache positions; selected historical memory receives a visible prefix mask. Memory- disabled generation delegates entirely to the original module.

5 Experimental Setup

Table 1 records the first checkpoint exactly. Tests use synthetic Qwen3 configs offline. The pre-trained run uses Python 3.10.11, PyTorch 2.12.1+CUDA 12.6, Transformers 4.55.4, eager attention, FP16, and an NVIDIA GeForce GTX 950M. The frozen model is not fine-tuned.

Table 1: Pinned Qwen checkpoint and PRA placement.

Checkpoint	Qwen/Qwen3-0.6B
Revision	c1899de289a04d12100db370d81485cdf75e47ca
Parameters / layers	596,049,920 / 28
Query heads / K/V heads / head width	16 / 8 / 128
RoPE base / advertised positions	10^6 / 40,960
PRA placement	layer 27 only
Initial gist / routing policy	mean, one gist/chunk, exact top- k
Native limit / direct / selected-memory cap	256 / 128 / 64 tokens
K/V residency	CPU full cache, selected transfer

Every run writes JSON and scalar CSV with the Git SHA, model revision, software versions, device, dtype, layer selection, limits, routing settings, timings, and metrics. The exact executed commit is retained in each artifact rather than inferred from the manuscript build.

6 Adapter Parity

PRA-disabled parity is the hard gate. The baseline is evaluated first; the same in-memory model is then wrapped, leaving every parameter object in place. Table 2 reports exact results for the five-token prompt “The capital of Portugal is” and four-token greedy decode.

Table 2: Original HF model versus memory-disabled PRA wrapper.

Metric	Result
Maximum / mean absolute logit difference	0 / 0
Maximum hidden-state difference	0
Greedy token agreement	100%
Greedy sequences exactly equal	yes
Generation-cache shapes equal	all 28 layers
Finite adapted logits	yes

Single-run prefill times were 0.447 s before wrapping and 0.136 s after wrapping. This order is confounded by first-call warm-up and is not a speed comparison. Exact outputs, rather than latency, are the parity evidence.

7 Native-KV Reference Smoke

An explicit 22-token reference about Lisbon is encoded through the frozen model. The selected layer captures [1, 8, 22, 128] K/V, stores it on CPU, creates a mean gist, and transfers all 22 tokens because the source fits one routing chunk. The physical transfer is 90,112 bytes, exactly $2 \times 22 \times 8 \times 128 \times 2$ bytes for FP16 K and V. The run reports 1.6 ms routing, 0.40 ms materialization orchestration, 0.15 ms selected transfer, and 0.11 ms combined attention. These synchronized phase values are instrumentation checks on one machine, not throughput claims. Cold reference construction took 0.144 s and the warm query took 0.191 s.

The generated continuation contains “Lisbon”, but the unmodified model also knows this fact. Consequently, this observation establishes functional selected-K/V transport and generation, not a

causal retrieval benefit. The exact native-replay claim is instead restricted to the synthetic tensor test, where GQA memory plus local K/V equals explicit dense composition.

8 Implicit Head and Bounded Long Prompt

A repeated prompt tokenizes to 408 logical tokens. The direct cap retains the final 128 tokens; the first 280 become `#_head`. That head is encoded in blocks of at most 64 tokens, with source-relative positions. The direct tail receives positions 280 through 407. Every encoding operation and the final selected-memory operation stays under the 256-token configured native limit; the largest observed source-encoding call is 64 tokens and violations are zero. The final finite-logit query took 0.258 s. This proves bounded execution and offset propagation for the frozen Qwen adapter. It does not show that the router selected the information needed by an arbitrary continuation.

9 Unrestricted QA Smoke

We next run one fixed HotpotQA example [10] and one QASPER example [3]. Four conditions use the same frozen checkpoint: question-only truncation, dense text left-truncated to 256 tokens, oracle evidence text-RAG, and PRA with the question direct and the complete source in native-K/V memory. Oracle text-RAG is an upper control because it receives annotated evidence. This two-example matrix diagnoses plumbing; it cannot estimate dataset accuracy.

Table 3: Unrestricted-QA smoke outcomes. Standard normalized token F1 is shown.

Dataset	Condition	F1	Annotated evidence selected
HotpotQA	truncation; dense; oracle text-RAG; PRA	0; 0; 0; 0	PRA: 0%
QASPER	truncation; dense; oracle text-RAG; PRA	0; 0.182; 0; 0.182	PRA: 0%

HotpotQA’s expected answer is “yes”; every condition begins with “No”. QASPER’s expected answer is “no”; dense and PRA generations begin with “No”, while truncation begins with “Yes”. However, PRA routes three tail chunks, none overlapping the annotated evidence, so the QASPER output cannot be credited to retrieved evidence. Qwen also fails the oracle text-RAG control on both examples. The proper conclusion is negative: this frozen top-layer, one-gist-per-chunk configuration has not demonstrated content-causal retrieval.

The selections expose a concrete failure mode. For HotpotQA, evidence lies at token spans 107–137 and 543–588, while routed chunks lie near 1216–1318. For QASPER, evidence lies at 0–317, while selected chunks lie near 4032–4192. Post-RoPE mean-key gists and an unadapted top-layer query systematically prefer irrelevant late-source chunks in these two probes.

10 Routing Representation Study

The smoke failure motivates a narrower test: is the post-RoPE representation appropriate for semantic routing? For token j at position p_j , the baseline pools

$$g_{\text{post}} = \frac{1}{m} \sum_{j=1}^m R(p_j) K_{\text{pre},j}, \quad (3)$$

which mixes content with different rotary phases. The position-neutral alternative pools $g_{\text{pre}} = m^{-1} \sum_j K_{\text{pre},j}$. A third representation pools the attention-input hidden states. These are hypotheses about ranking only; selected detail memory remains Qwen’s post-RoPE K and native V in every condition.

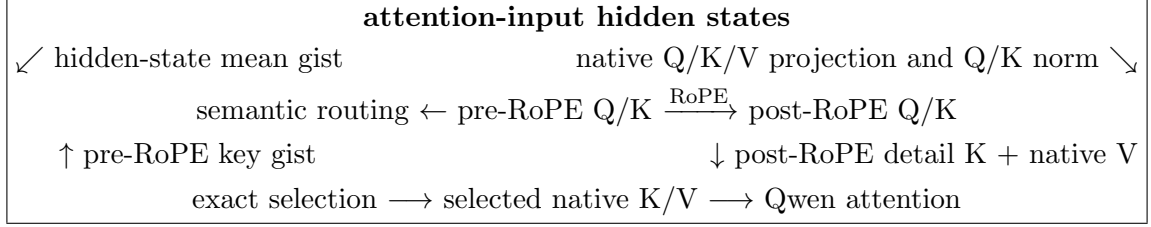


Figure 2: Routing and transport are separate. Pre-RoPE or hidden-state features rank chunks; the attention path always materializes post-RoPE native K/V.

10.1 Protocol

Stage 1 uses eight HotpotQA and eight QASPER validation examples, matched across representations. The frozen checkpoint, last-layer placement, one mean gist per chunk, 128-token encoding blocks, zero routing overlap, and exact tensorized search remain fixed. We compare post-RoPE keys, normalized pre-RoPE keys, and attention-input hidden states with 32-token routing chunks and $k \in \{3, 8, 16\}$. Evidence text is mapped to exact tokenizer spans. The final-token question query is captured at the same layer: GQA query groups are averaged into native K/V width for both key representations, while hidden-state routing uses the matched final hidden vector.

The experiment scores all chunks and does not generate answers. This keeps evidence ranking separate from decoding and from the hard materialization budget. Stage 2 carries the best two representations to 64- and 128-token chunks. Stage 3 confirms the best sparse configuration on two deterministic 32-example subsets with different seeds: 64 evaluations covering 61 unique examples. No model or router parameter is trained.

10.2 Representation and position bias

Table 4 reports Stage 1 means over both datasets. The selected fractions at $k = 3, 8, 16$ are 0.040, 0.107, and 0.213 for every representation. Post-RoPE scores are strongly correlated with later source position. Both position-neutral alternatives remove that correlation and improve evidence ranking.

Table 4: Stage 1 routing representation comparison, 16 matched examples and 32-token chunks.

Routing representation	Recall@3	Recall@8	Recall@16	MRR	Score-position r
Post-RoPE key mean	0.125	0.250	0.313	0.131	0.652
Pre-RoPE key mean	0.313	0.563	0.750	0.301	0.009
Hidden-state mean	0.438	0.500	0.750	0.276	-0.021

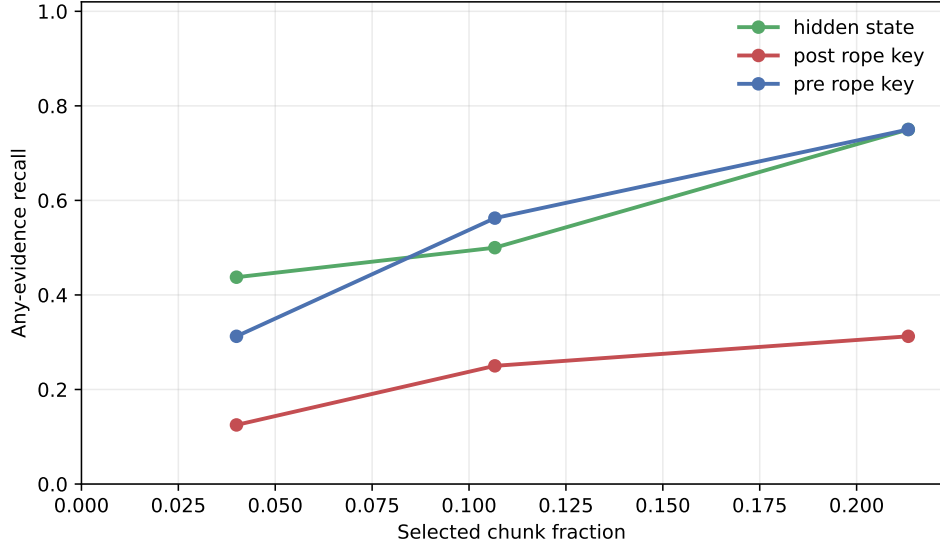


Figure 3: Any-evidence recall against routed chunk fraction in Stage 1. Pre-RoPE and hidden-state routing dominate the post-RoPE baseline, but high recall still requires broader selection.

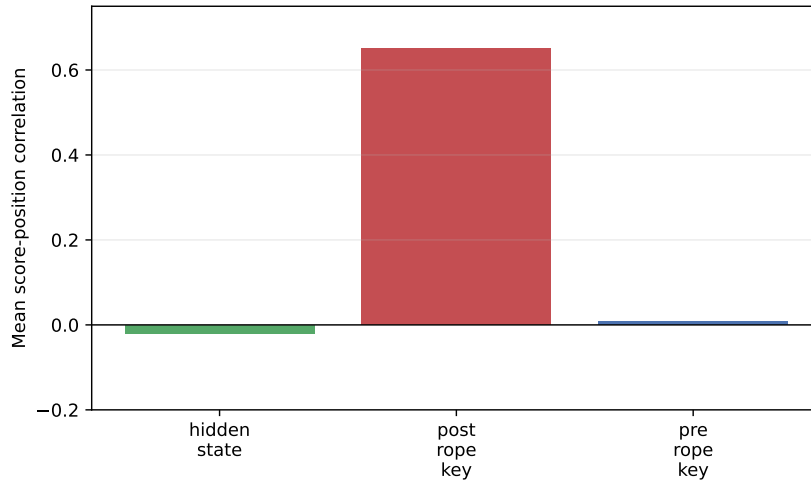


Figure 4: Mean chunk-score correlation with normalized source position. The late-source effect is specific to post-RoPE mean-key routing in this comparison.

The result supports the RoPE-contamination hypothesis: the post-RoPE representation is a poor mean-pooled semantic index. It does not establish that pre-RoPE keys are the best router. Hidden-state means provide the best low- k recall, while pre-RoPE keys provide the best MRR and recall@8. Native key space and semantic retrieval space need not coincide even before rotation.

10.3 Chunk size and breadth

Table 5 shows that larger chunks sometimes raise recall, but they also make a fixed k cover much more of the source. At 128 tokens and $k = 8$, both representations reach 0.688 recall while

selecting 41.3% of chunks. At $k = 16$, the selected fraction is 71.5%. These are useful boundary measurements, not sparse-routing wins.

Table 5: Stage 2 chunk-size sweep over the 16 matched examples. Fraction is reported at $k = 8$.

Representation	Chunk tokens	Recall@3	Recall@8	Recall@16	Selected fraction@8
Pre-RoPE key	32	0.313	0.563	0.750	0.107
Pre-RoPE key	64	0.250	0.438	0.813	0.211
Pre-RoPE key	128	0.375	0.688	0.875	0.413
Hidden state	32	0.438	0.500	0.750	0.107
Hidden state	64	0.313	0.625	0.813	0.211
Hidden state	128	0.438	0.688	0.813	0.413

The expected larger- k recall/cost tradeoff is confirmed. The predicted degradation of post-RoPE pooling with larger chunks is not tested after that representation failed Stage 1; the two finalists do not degrade monotonically. Their larger-chunk gains are inseparable from coarser evidence coverage and a larger selected source fraction.

10.4 Independent confirmation and gate

Hidden-state routing with 32-token chunks is selected for confirmation because it gives the best recall at $k = 3$. Table 6 combines both seeds. Materialization is capped at 128 tokens, so routed breadth continues to grow at $k = 8$ and 16 while the physically materialized fraction stays near 0.059.

Table 6: Hidden-state confirmation over 64 evaluations. Intervals are 95% Wilson intervals.

k	Any recall	95% interval	All recall	Selected fraction	Materialized fraction
3	0.391	[0.281, 0.513]	0.063	0.045	0.045
8	0.578	[0.456, 0.691]	0.203	0.119	0.059
16	0.797	[0.683, 0.877]	0.500	0.238	0.059

At $k = 3$, HotpotQA recall is 0.625 but QASPER recall is 0.156; combined MRR is 0.326. The combined score-position correlation is -0.077 , so the original late-position failure is not the remaining blocker. The predeclared promotion gate requires combined recall@3 ≥ 0.70 , each dataset ≥ 0.60 , MRR ≥ 0.50 , selected fraction ≤ 0.10 , and absolute position correlation ≤ 0.15 . Only the last two conditions pass. Qwen therefore does not advance to Llama.

One hidden-state gist adds 1.57% over detail K/V bytes at 32-token chunks. On the GTX 950M, mean packed-index construction is 3.50 ms, warm exact routing plus selection is 1.90–2.74 ms, and the isolated `torch.topk` primitive is about 0.05 ms. These are single-request Python measurements, not serving-throughput claims.

11 Systems Observations

The first run reaches 1,345,618,432 allocated GPU bytes, including the complete FP16 model and temporary activations. Full reference K/V remains on CPU; only selected detail K/V moves to the GPU. This is evidence that the residency path executes, not a memory-savings comparison: no matched dense long-context peak is measured here. Likewise, cold source construction, warm routing, generation, and transfer are reported separately so cache reuse is not hidden.

The current HF path is intentionally eager and unfused. Variable memory rows are padded into a rectangle before Qwen’s native eager function. Flash attention, SDPA, custom kernels, ANN routing, asynchronous overlap, and serving concurrency remain outside this correctness phase.

12 Limitations and Next Experiments

The study has one family, one 596M-parameter checkpoint, one PRA layer placement, and one-gist zero-parameter routing. The representation/chunk sweeps use 16 matched examples; confirmation uses two deterministic subsets rather than a full validation corpus, and three examples overlap between seeds. The evaluation measures annotated-span ranking, not answer EM/F1, because a model cannot receive credit for an answer when selected K/V excludes the evidence. There is no matched learned text retriever, multi-layer placement study, LoRA adaptation, serving-throughput study, or full dense-context quality comparison.

The next intervention should be a small evidence-supervised routing or gist adapter with the base Qwen checkpoint frozen. It should preserve hidden-state and pre-RoPE baselines, report parameter count and training cost, and test cross-dataset generalization. Multi-gist subchunk means are a reasonable zero-parameter diagnostic, but the present result does not justify a broad gist-count sweep: the dominant sparse failure is semantic ranking on QASPER. Llama remains blocked until Qwen passes the declared recall/MRR/sparsity/position gate. Gemma remains later.

13 Related Work

PRA differs from text retrieval-augmented generation, which retrieves text and lets the model encode it in the current prompt [5]. It retrieves layer-native K/V after a separate bounded encoding path. This makes PRA complementary to RAG: a text retriever may choose documents before PRA routes chunks inside cached model state. Sparse-attention systems such as Longformer and BigBird constrain token-token connectivity inside a forward pass [2, 11]; PRA instead materializes selected state from an external addressable cache. KV-cache systems emphasize serving capacity and token retention policies [4, 12, 6]. Paper 2 asks whether selection can be inserted beneath an existing pretrained family without changing its disabled behavior.

14 Conclusion

One shared PRA core now serves both the controlled model and a thin Qwen adapter. The frozen Qwen3-0.6B checkpoint passes bit-exact disabled parity, preserves native RoPE and GQA cache layout, replays selected native K/V through generation, and executes an implicit long-prompt head under a hard native-operation bound. The routing study adds a second architectural result: the representation appropriate for native positional attention is not necessarily appropriate for semantic selection. Pre-RoPE and hidden-state gists remove the strong late-position effect without changing transported post-RoPE K/V.

That correction is necessary but insufficient. Hidden-state routing reproduces useful evidence ranking above the post-RoPE baseline, yet sparse recall and MRR remain below the declared gate, especially on QASPER. Selecting nearly one quarter of chunks raises recall@16 to 0.797, but does not constitute a sparse-routing solution. The correct next step is a small frozen-Qwen learned router, not Llama integration and not a claim of solved long-context retrieval.

A Code-Level Adapter Reimplementation Guide

This appendix gives the minimum code structure needed to reproduce the integration without copying the Qwen implementation. Source references use physical line numbers at repository commit 9732da1. This checkpoint implements the three routing representations used above. The line references are deliberately revision-pinned because upstream Transformers APIs and local diagnostics will move over time.

A.1 Tensor and ownership contract

A new family should add projection, position, cache, mask, kernel, and output conventions in its adapter, while leaving selection and memory policy in the shared core.

The canonical tensor contract is

$$Q \in \mathbb{R}^{B \times H_q \times T_q \times d_h}, \quad K, V \in \mathbb{R}^{B \times H_{kv} \times T_k \times d_h}. \quad (4)$$

Reference entries use $B = 1$ and keep H_{kv} , including for GQA or MQA. They are not expanded to H_q in persistent storage. The controlled replay implementation shows the only required expansion point in `memory_batching.py`, lines 216–290: after memory and local K/V are joined, native K/V heads may be repeated for the attention computation. Qwen’s installed eager kernel performs the corresponding operation in the production adapter.

A.2 Minimal forward-path skeleton

The following pseudocode is the adapter’s essential control flow. It is intentionally family-neutral; “native” operations must call the installed model implementation rather than reimplementing its mathematics.

```
01 forward(hidden, native_positions, native_mask, past):
02     if capture_is_armed:
03         q_pre, k_pre, v = native_project_and_norm(hidden)
04         q_post, k_post = native_position(q_pre, k_pre)
05         save_transient_routing_features(hidden, q_pre, q_post, k_pre)
06         save_post_position_detail_kv(k_post, v)
07
08     if memory_is_disabled or reference_cache_is_empty:
09         return original_attention(hidden, native_positions,
10                                 native_mask, past)
11
12     q_pre, k_pre, v = native_project_and_norm(hidden)
13     q_post, k_post = native_position(q_pre, k_pre)
14     if past is not None:
15         k_post, v = past.update(k_post, v, layer_id, native_kwargs)
16     route_q = matched_query(hidden, q_pre, q_post)
17     memory = shared_core.prepare_memory(q_post, route_q,
18                                       direct_tokens=k_post.shape[2])
19     if memory.is_empty:
20         return native_attention_and_output(q_post, k_post, v, native_mask)
21     k, v, mask = shared_core.combine_local_and_memory_kv(
```

```

22         k_post, v, memory, native_mask, query_tokens=q_post.shape[2])
23     return native_attention_and_output(q_post, k, v, mask)

```

Lines 8–10 are a correctness boundary, not an optimization. The Qwen implementation delegates to the untouched module at `hf/qwen.py`, lines 187–198, which makes disabled behavior auditable and bit-exact. In the active path, projection and RoPE occur at lines 200–203 and `Cache.update` occurs once at lines 204–211. Lines 212–217 choose the matched routing query without altering the post-RoPE attention query. If routing returns no materialized memory, lines 220–230 call the native kernel directly. Delegating at that point would execute a second cache update and duplicate generation history.

Memory is composed only after the native cache update. The shared operation at `core.py`, lines 362–404, pads variable memory rows, prepends memory K/V, creates an all-visible additive mask for valid memory positions, masks padding, and concatenates the unchanged local mask. The adapter then invokes Qwen’s eager function and one pretrained `o_proj` at `hf/qwen.py`, lines 89–107 and 242–254. Thus local and memory positions share one softmax and one output projection.

A.3 Publishing a reference

Reference construction is a bounded prefill, not a second attention mechanism. A portable implementation follows this recipe:

```

01  disable_PRA_memory_during_reference_encoding()
02  for [block_start:block_end] in bounded_encoding_blocks(source):
03      position_ids = logical_source_positions(block_start, block_end)
04      arm_post_position_kv_capture(position_ids)
05      model(block_ids, position_ids=position_ids, use_cache=False)
06      captured = consume_native_kv_for_each_PRA_layer()
07      for routing_window in overlapping_windows(captured):
08          gist = compute_routing_gists(routing_window.native_kv)
09          publish(uri, logical_offsets, routing_window.native_kv, gist)
10  restore_previous_memory_state()

```

The concrete loop is `hf/injection.py`, lines 127–204. Encoding block size limits one native model invocation; routing chunk size and overlap independently determine index granularity (lines 144–151). Each chunk slices the captured post-RoPE K/V, chooses transient routing features, computes one or more compact gists, and records logical source offsets (lines 152–197). Keeping those two scales separate is important: changing index granularity must not silently change how the pretrained model encoded the source.

The same operation implements displaced long-prompt history. At `hf/injection.py`, lines 206–218, the prefix is published under the reserved `#_head` URI and the direct tail retains continued logical position IDs. Every reference-encoding call and every direct call must satisfy the model-safe native bound even when the logical source is longer.

A.4 Porting another decoder family

A practical port begins by locating seven hooks in the installed family: the attention module inside each decoder layer; Q/K/V projections; any Q/K normalization; positional encoding; generation-cache update and its keyword arguments; native mask convention; and the eager attention plus output projection. Implement the six abstract methods declared at `hf/adapter_base.py`, lines

85–106, then add a narrow type-selection branch analogous to `inject_pra` at `hf/injection.py`, lines 228–253. No checkpoint conversion or parameter copy should be needed.

The recommended validation order is:

1. With memory disabled, require exact logits, hidden states, greedy tokens, and generation- cache lengths against an unwrapped copy (test lines 55–72).
2. Capture a short reference and verify post-position state, logical offsets, device residency, and H_{kv} rather than H_q storage (lines 74–93).
3. Verify matched pre/post capture, representation switching, unchanged post-RoPE detail K/V, GQA grouping, serialization, and device parity (lines 95–171).
4. Compare GQA replay with explicit head repetition on a synthetic tensor (lines 173–196).
5. Exercise an over-limit logical prompt and verify bounded native operations plus continued position IDs (lines 198–220).
6. Generate with active memory and assert that direct cache length grows by exactly one per decoded token (lines 222–237).

Four mistakes deserve explicit tests. Applying position encoding again to stored post-position keys changes their basis. Permanently expanding GQA K/V multiplies memory without adding information. Replacing the family mask with a generic causal mask can alter padding or sliding-window behavior. Finally, calling the wrapped module after manually updating its cache appends the same local token twice. Passing transport and parity tests establishes a faithful adapter; it does not establish that the router selects causally useful evidence, which requires the separate quality controls reported in the main paper.

A.5 Revision-pinned source map

Table 7 connects the preceding control flow to the implementation.

Table 7: Source map for an independent adapter implementation.

Responsibility	File and physical lines
Abstract family boundary	<code>src/pras_torch/hf/adapter_base.py</code> , 16–106
Qwen projections, RoPE, native kernel, and output	<code>src/pras_torch/hf/qwen.py</code> , 50–107
Matched routing/detail capture	<code>src/pras_torch/hf/qwen.py</code> , 109–148; <code>src/pras_torch/hf/injection.py</code> , 76–204
Disabled and active runtime paths	<code>src/pras_torch/hf/qwen.py</code> , 168–272
Routing query and GQA reduction	<code>src/pras_torch/core.py</code> , 92–121
Budget, deduplication, transfer, and materialization	<code>src/pras_torch/core.py</code> , 155–360
K/V and additive-mask composition	<code>src/pras_torch/core.py</code> , 362–404
Selected-layer injection	<code>src/pras_torch/hf/injection.py</code> , 228–253
Parity, routing, GQA, head, cache, and selection gates	<code>tests/test_hf_integration.py</code> , 55–252

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: <https://arxiv.org/abs/2105.03011>.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [7] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [8] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. Preprint: <https://arxiv.org/abs/1706.03762>.
- [10] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. Preprint: <https://arxiv.org/abs/1809.09600>.
- [11] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [12] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.